

Faculty of Engineering

Internal Design Document

# MQTT Protocol Design for Distributed Real-Time Telemetry System for QUT Microgrid


**Date:** August 28, 2026

**Student Engineer:** Dylan Hessing

**Student ID:** n11245310

**Supervisor:** Assoc. Prof. Geoff Walker


**Version History**

| Version | Date | Author | Changes / Comments |
|---|---|---|---|
| 1 | April 20, 2026 | Dylan Hessing | Initial Design |

# Contents

**1 Overview** **2**

**2 Scope** **2**

**3 Broker Selection & Configuration** **2**

**4 Topic Hierachy** **3**

4.1 Naming Scheme . . . . . 3

4.2 Example Topics . . . . . 4

4.3 Wildcard Subscription Patterns . . . . . 4

**5 Payload Format** **4**

5.1 Format Selection . . . . . 4

5.2 Single-Sample Payload . . . . . 4

5.3 Burst Payloads . . . . . 5

**6 Quality of Service Levels & Retained Messages** **5**

6.1 Quality of Service Strategy . . . . . 5

6.2 Retained Messages . . . . . 6

6.3 Last Will and Testament . . . . . 6

**7 Client Architecture** **6**

7.1 DAQ Nodes (STM32 Publishers) . . . . . 6

7.2 Historian Bridge . . . . . 7

7.3 Security . . . . . 8

# 1 Overview

This document describes the MQTT protocol design for the distributed real-time telemetry system developed as part of the EGH490 thesis project at the Queensland University of Technology. The system acquires electrical measurments from STM32-based data acquisition nodes deployed across the QUT microgrid and transports them to centralised time-series historian via an MQTT message broker,

The design covers broker selection and configuration, topic heirachy, payload format, quality of service strategy, client architecture, security posture, and historian integration. All decisions are scoped to a two-node validation testbed operating over a wired ethernet LAN, with the architecture designed to generalise to an arbitrary number of nodes without structural change.

# 2 Scope

The following contraints apply to all documented design decisions.

Table 1: Design Constraints

| Parameter | Value |
|---|---|
| Transport Medium | Wired Ethernet (LAN) only |
| Number of DAQ nodes (validation testbed) | 2 |
| DAQ node MCUJ | STM32 (IEEE 1588 hardware timestamping supported) |
| Time synchronisation protocol | PTP (IEEE 1588-2019) |
| Historian | InfluxDB v2 |
| Target measurement standards | IEC 61000-4-30 Class S, IEEE 1459-2010 |

Wireless transport is specifically out of scope. The wired only constraint eliminates RF-induced jitter as a variable in timestamp accuracy, permitting PTP synchronisation to operate within the sub-microsectond range achievable on a switched Ethernet topology. The two-node testbed reflects hardware availabilityat the time of validation; the architecture imposes no upper bound on the number of nodes.

# 3 Broker Selection & Configuration

Eclipse Mosquitto is selected as the MQTT broker. This is suitable of a research testbed with two DAQ nodes as it is:

- lightweight
- well documented
- suitable for co-deployment on the historian server alongside InfluxDB without significant resource contention
- carries no licensing contraintes under the eclipse public licence

Alternatives such as EMQX and HiveMQ were considered but are designed for large-scale deployments with thousands of concurrent clients; the additional operational complexity is not justified at this scale.

The broker runs on the microgrid historian server, bound to the lab LAN interface. Key configuration parameters are as follows:

```
listener 1883 <lab_lan_ip>
allow_anonymous false
password_file /etc/mosquitto/passwd

persistence true
persistence_location /var/lib/mosquitto/

max_inflight_messages 100
max_queued_messages   1000

log_type error
log_type warning
log_type notice
```

Port 1883 is used for internal lab LAN traffic. TLS on port 8883 is reserved as a future hardening measure and discussed in section X. Persistence is enabled so that QoS 1 messages survive a broker restart without loss. The `max_inflight_messages` and `max_queued_messages` parameters are bounded to prevent unbounded memory growth under a backlogged subscriber.

# 4 Topic Hierachy

## 4.1 Naming Scheme

Topics follow a four-level hierachy:

```
microgrid/<node_id>/<subsystem>/<measurement>
```

Table 2: Hierachy Description

| Segment | Description | Example Values |
|---|---|---|
| `node_id` | Fixes, zero-padded node identifier | `node_1`, `node_2` |
| `subsystem` | Electrical quantity category | `voltage`, `current`, `power`, `frequency` |
| `measurement` | Specific measured quality | `phase_a`, `active`, `reactive`, `fundamental` |

Node identifiers are fixed strings assigned at firmware flash time. MAC addresses and IP addresses are explicitly avoided as node identifiers as both are mutable and would invalidate historical records if hardware is replaced. Measurement names are aligned with IEEE 1459-2010 terminology where applicable

### 4.2 Example Topics

### 4.3 Wildcard Subscription Patterns

Table 3: Wildcard Subscription Patterns

| Pattern | Use Case |
|---|---|
| microgrid/# | full subscription: used by the InfluxDB historian bridge |
| microgrid//status/heartbeat | node liveness monitoring across all nodes |
| microgrid/node_01/# | All measurements from a single node (debugging) |
| microgrid//voltage# | All voltage measurements across all nodes |

## 5 Payload Format

### 5.1 Format Selection

Payloads are serialised as UTF-8 JSON. This prioritises developer ergonomics and debuggability during the research phase: payloads are huiman-readable, trivially inspected with any MQTT client and require no schema negotiation between publisher and subscriber. Migration to a binary format such as MessagePack or CBOR is possible if bandwidth or parsing overhead becomes a concern, but is not anticipated on a wired LAN with two nodes.

### 5.2 Single-Sample Payload

```
{
  "ts": 1713400000123456000,
  "v": 231.45
}
```

Table 4: Single-Sample Payload Description

| Field | Type | Description |
|---|---|---|
| `ts` | uint64 | PTP-synchronised Unix timestamp (fractional seconds) sourced from the STM32 IEEE 1588 hardware clock |
| `v` | float32 | Measurement value in SI units |

The timestamp is sourcef from the STM32 IEEE 1588 hardware timestamping peripheral, not the system tick counter. This preserves PTP synchronisation accurcy end–to-end, from the ADC sample through to the broker and into the InfluxDB point timestamp

Units are implied by the topic rather than included in the payload. For example, a message on `microgrid/node_01/volt` is understood to carry a value in volts.

### 5.3 Burst Payloads

For high-frequency acquisition where per-message overhead is a concern, multiple samples may be batched into a single publish:

```
{
  "ts": 1713400000123456000,
  "dt": 0.000312,
  "v":  [231.45, 231.47, 231.43, 231.50]
}
```

Table 5: Burst Payload Description

| Field | Type | Description |
|---|---|---|
| `ts` | float64 | Timestamp of the first sample in the batch |
| `dt` | float64 | Inter-sample interval in seconds |
| `v` | float32[] | Ordered array of sample values |

The historian bridge reconstructs individual timestamps as $t_s + i \cdot \Delta t$ for sample index i, and writes each as a separate InfluxDB point.

The decision between single–sample and burst sample publishing depends on the ADC sample rate (yet to be finalided). This should be resolved before the firmware publish loop is implemented.

## 6 Quality of Service Levels & Retained Messages

### 6.1 Quality of Service Strategy

MQTT defines three quality of service (QoS) levels governing delivery guarantees between publisher and broker, and between broker and subscriber. The appropriate level is selected per message type based on frequency and the consequence of loss.

Table 6: QoS Strategy

| Message Type | QoS | Rationale |
|---|---|---|
| Periodic Telemetry Samples | 0 (fire and forget) | High Frequency; ACK overhead is disproportionate to benefit; a missed sample is bounded and tolerable; retransmitting a stale sample is actively harmful to the historian. |
| Events and Alarms | 1 (at least once) | Low frequency; loss of a discrete event is significant |
| Configuration updates | 1 (at least once) | Low frequency; idempotent processing at the reciever |

QoS 2 (exactly once) is not used as the four-way handshake overhead is rarely justified in practice. Additionally, the historian bridge handles potoential duplicate ingestion gracefully.

### 6.2 Retained Messages

The MQTT retained message flag instructs the broker to store the most recent messages on a given topic and deliver it immediately to any new subscriber. This is used selectively

Table 7: Retained Messages

| Topic | Retained | Rationale |
|---|---|---|
| `microgrid/<node_id>/status/heartbeat` | Yes | New subscribers immediately see current node liveness without waiting for the next publish. |
| `microgrid/<node_id>/status/config` | Yes | Current node configuration is available to sbscribe on connect. |
| All telemetry topics | No | Retaining high–frequency samples causes a burst of stale data delivered to every new subscriber connection. |

### 6.3 Last Will and Testament

Each DAQ node client is configured with a Last Will and Testament (LWT) message. The LWT is a payload pre-registered with the broker at connection time; the broker publishes it automatically if the client disconnects ungracefully without sending a disconnect packet.

Table 8: Last Will and Testament

| Parameter | Value |
|---|---|
| Topic | `microgrid/<node_id>/status/heartbeat` |
| Payload | `{"status": "offline"}` |
| QoS | 1 |
| Retain | true |

On reconnection, the node publishes `{"status": "online"}` to the same topic (retained, QoS 1), overwriting the LWT. This gives the broader system a reliable, broker-mediated mechanism for node liveness detection without requiring any application-level timeout logic in the subscriber.

## 7 Client Architecture

### 7.1 DAQ Nodes (STM32 Publishers)

Each STM32 DAQ node operates as an MQTT publisher. The node runs a lightweight MQTT client library on top of the STM32 HAL and LwIP TCP/IP stack. Two candidate libraries are suitable for this environment:

Table 9: DAQ Nodes

| Library | Notes |
|---|---|
| Paho MQTT Embedded C | Mature, widely used, runs on bare metal or FreeRTOS |
| FreeRTOS CoreMQTT | Purpose-built for embedded, minimal footprint, well-suited if FreeRTOS is adopted as the RTOS |

Final Library selection is dependent on the RTOS decision for DAQ firmware which is outside the scope of this document.

**Publish Loop**

The nominal publish sequence on each node is as follows:

1. ADC DMA transfer completes; the ISR signals the processing task via a FreeRTOS notification or baremetal semaphore
2. The processing task computes any required derived quantities
3. The result is serialised into a JSON payload as described
4. The payload is published to the broker on the appropriate topic at QoS 0

**Connection Management**

On power-on, the node waits for network availability then connects to the broker using its pre-configured credentials. On connection loss, exponential back–off is applied before reconnect attempts. Critical events such as alarms and status changes are buffered in a SRAM ring buffer during disconnection and flushed on reconnect. Periodic telemetry samples are not buffered during disconnection; the resulting gap is visible in hte historian which is acceptable for a research testbed.

Client IDs follow the convention `mqttc--<node_id>`.

## 7.2 Historian Bridge

Telegraf with the MQTT consumer input plugin is used as the historian bridge. This provides a configuration-driven pathway from the broker to InfluxDB with no custom subscriber code required. The `topic_parsing` feature extracts topic path segments directly as InfluxDB tags, so each point written to InfluxDB is automatically tagged with `node_id`, `subsystem` and `measurement`.

```
[[inputs.mqtt_consumer]]
  servers     = ["tcp://localhost:1883"]
  topics      = ["microgrid/#"]
  username    = "telegraf"
  password    = "${TELEGRAF_MQTT_PASSWORD}"
  qos         = 0
  data_format = "json"
  json_time_key    = "ts"
  json_time_format = "unix_ns"

  [[inputs.mqtt_consumer.topic_parsing]]
```

```
    topic = "microgrid/<node_id>/<subsystem>/<measurement>"
    tags  = "_/<node_id>/_/<measurement>"

[[outputs.influxdb_v2]]
  urls         = ["http://localhost:8086"]
  token        = "${INFLUXDB_TOKEN}"
  organization = "qut-microgrid"
  bucket       = "telemetry"
```

### 7.3 Security

Security is implemented in phases dependent on deployment environment, The system operates on a university reseach lab LAN and is not exposed to public internet; accordingly, the security measures are pragmatic rather than exhaustive.

**Phase 1: Development**

- The broker is bound to the lab LAN interface only and is not reachable from the broader campus network.
- Anonymous connections are disbaled
- Username and password authentication is enabled via Mosquitto's built–in password file mechanism
- Seperate credentials are issued per client (one per DAQ node, one for the Telegraf bridge)

**Phase 2: Hardening**

Access control lists (ACLs) enforce the principle of least privege. Each DAQ node may only publish to its own topic subtree; a compromised node cannot spoof measurements from antoher node. The Telegraf bridge may subscribe to the full topic tree but cannot publish.

**Phase 3: TLS (future)**

If the broker is ever exposed outside the lab VLAN, mutual TLS on port 8883 with certificates issued by a local certificate authority is the intended upgrade path. This is not required for the current testbed scope, however has been included should the system be deployed more broadly.